Hashing

Arman Ferdous

What is Hashing?

- Compressing a large set of objects into a small set of objects.
- An item x gets mapped into H(x).
- Collisions will definitely occur.
- In practice we can avoid collisions with high probability.

String Hashing

- Useful to test whether two strings are equal.
- Can also test whether any two substrings of length k are equal.
- How to do it? Imagine the string as a number of a higher base.

Code

```
const int B = 420;
const int M = 1e9+7;

ll H (string &s) {
    ll h = 0;
    for (int i = 0; i < sz(s); ++i) {
        h = h * B % M;
        h = (h + s[i]) % M;
    }
    return h;
}

int main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);

    string s, t;
    cin >> s >> t;

    cout << H(s) << " " << H(t) << "\n";
    return 0;
}
```

Hash Collision Chance

- Suppose you use random base B and fixed modulo M.
- Then for two strings of length n, their hash will collide with probability

$$(n - 1) / (M - 1)$$

How to calculate hash of every substring fast

- We need a way to compute hash of any substring of a string fast.
- Use prefix hashing.

Code

```
const int N = 1e5+10;
const int B = 420;
const int M = 1e9+7;

ll b[N], h[N];

void precalc (string &s) {
    h[0] = 0;
    b[0] = 1;
    for (int i = 1; i < sz(s); ++i) {
        h[i] = ((h[i - 1] * B % M) + s[i]) % M;
        b[i] = b[i - 1] * B % M;
    }
}

ll calc (int l, int r) {
    return (h[r] - (h[l - 1] * b[r - l + 1] % M) + M) % M;
}

int main() {
    ios_base::sync_with_stdio(0); cin.tie(0); cout.tie(0);

    string s;
    cin >> s;

    s = " " + s;
    precalc(s);

    for (int i = 1; i < sz(s); ++i) {
        for (int j = i; j < sz(s); ++j) {
            ll res = calc(i, j);
            cout << i << "," << j << ": " << res << "\n";
        }
```

Application 1

- Given a string S and a string T, determine whether T occurs as a substring inside S.

Application 2

- Given a string S, calculate the number of unique substring of S.

Application 3

- Given a string S, and Q queries:
  L R: determine whether the substring S[L…R] is a palindrome

Application 4

- Given a string S, queries:

  i j: Determine the length of the matching prefix of the strings S[i…] and S[j…]

Application 5

- Given a string S, multiple queries:

  L1 R1 L2 R2: what is the lexicographical order S[L1…R1] of S[L2…R2].

Application 6

- Given a string S, count the total number of palindromic substrings of S.

Application 7

- Given a string S, compute the period of S. In other words, highest value of k such that:

  S = T + T + … + T (k times)

Set Hash

- Suppose we need to test equality of two sets. Now order doesn’t matter.
- Here we can use XOR hashing with random weights.

Application 1

- You have n sets, initially empty. Each query an element randomly gets inserted or deleted from a set. Also you need to test whether two sets are equal.

Application 2

- Given a graph, two nodes have the same neighborhood if they have the same outgoing edges. Determine how many pairs of same neighborhood nodes are there.

Multiset Hashing

- How can we incorporate duplicate items? We can easily modify our standard rolling hash.

$$H(A) = (w(x1) + w(x2) + w(x3) + \ldots) \% M$$
$$H(A) = (w(x1) * w(x2) * w(x3) * \ldots) \% M$$

Problem

- Given n bounded rectangles, two people are in the same region if they are inside same rectangles.
- Rectangles can be nested.
- Test multiple query points whether they are in same region.

Tree Hashing

- Given two trees determine whether they are structurally same.
- This has two versions, rooted and unrooted.

Dynamic String Hashing

- We want to calculate hash of substrings, but the characters will keep changing too.